Estruturas de Dados Básicas
Pilha & Fila

Estruturas de Dados



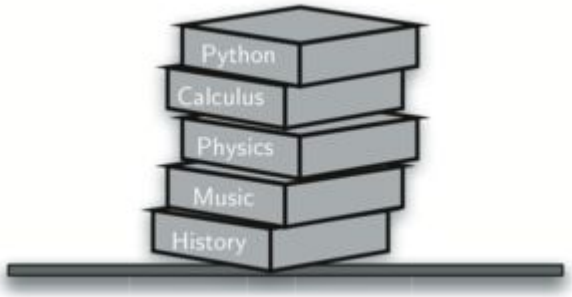
Universidade de Brasília
Departamento de Ciência da Computação

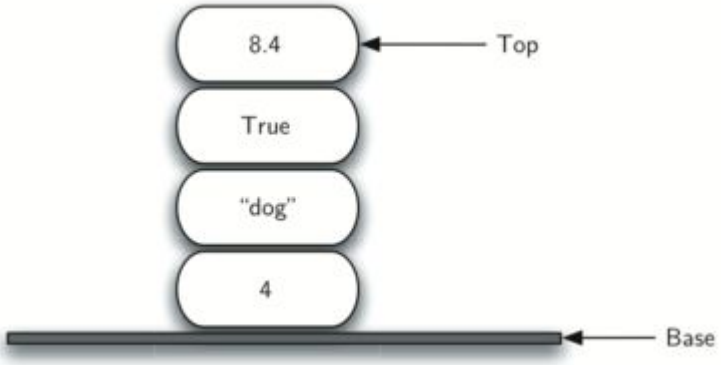
Estruturas lineares

- Os itens são ordenados de acordo com ordem que são inseridos ou removidos da estrutura.
- Uma vez que um item é inserido, ele fica em uma mesma posição em relação aos demais.
- O que distingue uma estrutura linear é a maneira em que itens são inseridos e removidos.

Pilha

Uma pilha (stack) é uma coleção ordenada de itens onde a inserção de novos itens e a remoção de itens existentes sempre ocorrem na mesma extremidade. Esta extremidade é comumente chamada de topo.





LIFO, último a entrar primeiro a sair (last in first out)

Pilha

Implemente uma pilha.

Tipo Abstrato de Dado

- O Tipo Abstrato de Dado (TAD) é uma especificação de um conjunto de dados e operações que podem ser executadas sobre esses dados.
- O TAD tem uma **especificação** e uma **implementação**: a separação permite o uso do TAD sem conhecer nada sobre a sua implementação.
- Especificação: Sintaxe e semântica
- Implementação: Estrutura de dados e algoritmos

Tipo Abstrato de Dados Pilha

- `Stack()` cria uma nova pilha vazia.
- `push(item)` insere um novo `item` na pilha.
- `pop()` remove o item que está no topo da pilha.
- `peek()` retorna o item no topo mas não o remove da pilha.
- `isEmpty()` testa se a pilha está vazia.
- `size()` retorna o número de item na pilha.

Tipo Abstrato de Dados Pilha

Implemente uma pilha usando TAD.

Tipo Abstrato de Dados Pilha

```
class Stack:
    def __init__(self):
        self.items = []

    def isEmpty(self):
        return self.items == []

    def push(self, item):
        self.items.append(item)

    def pop(self):
        return self.items.pop()

    def peek(self):
        return self.items[-1]

    def size(self):
         return len(self.items)
```

Tipo Abstrato de Dados Pilha

```
class Stack:
    def __init__(self):
        self.items = []

    def isEmpty(self):
        return self.items == []

    def push(self, item):
        self.items.append(item)

    def pop(self):
        return self.items.pop()

    def peek(self):
        return self.items[-1]

    def size(self):
         return len(self.items)
```



```
from pythonds.basic.stack import Stack
```

Tipo Abstrato de Dados Pilha

```
class Stack:
    def __init__(self):
        self.items = []

    def isEmpty(self):
        return self.items == []

    def push(self, item):
        self.items.append(item)

    def pop(self):
        return self.items.pop()

    def peek(self):
        return self.items[-1]

    def size(self):
         return len(self.items)
```

```
from pythonds.basic.stack import Stack
```

Insere e remove no final da lista - Operação O(1)

Tipo Abstrato de Dados Pilha

```
class Stack:
    def __init__(self):
        self.items = []

    def isEmpty(self):
        return self.items == []

    def push(self, item):
        self.items.append(item)

    def pop(self):
        return self.items.pop()

    def peek(self):
        return self.items[-1]

    def size(self):
         return len(self.items)
```

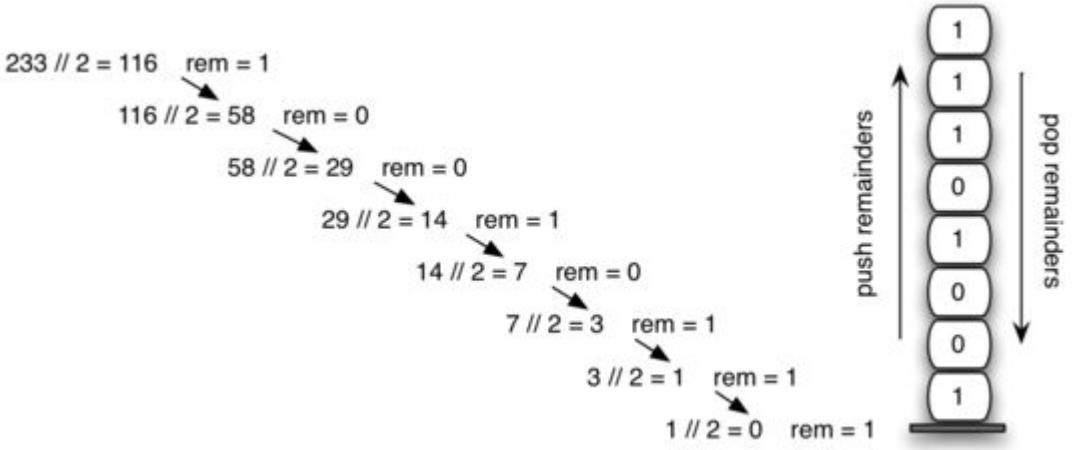
```
from pythonds.basic.stack import Stack
```

Insere e remove no final da lista - Operação O(1)

E se a pilha estiver vazia?

Conversão de Decimal para Binário

- Exemplo: $233_{10} = 11101001_2$
- Algoritmo chamado de “Divisão por 2” que usa uma pilha para manter os dígitos da representação binária.



Conversão de Decimal para Binário

```
from pythonds.basic.stack import Stack

def divideBy2(decNumber):
   remstack = Stack()

   while decNumber > 0:
       rem = decNumber % 2
       remstack.push(rem)
       decNumber = decNumber // 2

   binString = ""
   while not remstack.isEmpty():
       binString = binString + str(remstack.pop())

   return binString

print(divideBy2(42))
```

Conversão de Decimal para Binário

```
from pythonds.basic.stack import Stack

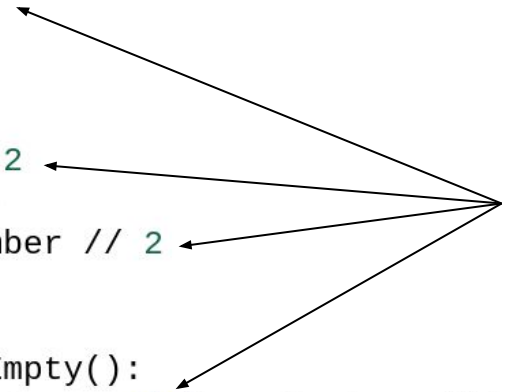
def divideBy2(decNumber):
    remstack = Stack()

    while decNumber > 0:
        rem = decNumber % 2
        remstack.push(rem)
        decNumber = decNumber // 2

    binString = ""
    while not remstack.isEmpty():
        binString = binString + str(remstack.pop())

    return binString

print(divideBy2(42))
```



`divideBy(number, base)`
Adicionar string de dígitos "0123456789ABCDEF" para Binário, Octal e Hexadecimal

Fila

Uma fila (queue) é uma coleção ordenada de itens em que a inserção de novos itens acontece em uma extremidade, chamado de "fim" (rear), e a remoção de itens existentes ocorre no outro extremo, comumente chamado de "início" (front).



FIFO, primeiro a entrar, primeiro a sair (first-in first-out)

Tipo Abstrato de Dados Fila

- `Queue()` cria uma nova fila que está vazia.
- `enqueue(item)` insere um novo `item` no final da fila.
- `dequeue()` remove o item do início da fila.
- `isEmpty()` testa para ver se a fila está vazia.
- `size()` retorna o número de itens na fila.

Tipo Abstrato de Dados Fila

Implemente uma fila usando TAD.

Tipo Abstrato de Dados Fila

```
class Queue:
    def __init__(self):
        self.items = []

    def isEmpty(self):
        return self.items == []

    def enqueue(self, item):
        self.items.insert(0, item)

    def dequeue(self):
        return self.items.pop()

    def size(self):
         return len(self.items)
```

Tipo Abstrato de Dados Fila

```
class Queue:
    def __init__(self):
        self.items = []

    def isEmpty(self):
        return self.items == []

    def enqueue(self, item):
        self.items.insert(0, item)

    def dequeue(self):
        return self.items.pop()

    def size(self):
         return len(self.items)
```

```
from pythonds.basic.queue import Queue
```

Tipo Abstrato de Dados Fila

```
class Queue:
    def __init__(self):
        self.items = []

    def isEmpty(self):
        return self.items == []

    def enqueue(self, item):
        self.items.insert(0, item)

    def dequeue(self):
        return self.items.pop()

    def size(self):
         return len(self.items)
```

```
from pythonds.basic.queue import Queue
```

Insere no início da lista - O(n)
Remove no fim da lista - O(1)

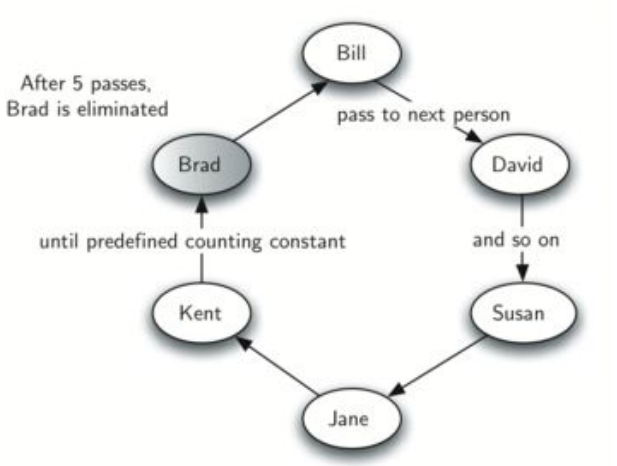
Fila de Prioridade

Implemente uma fila de prioridade para pessoas considerando que.

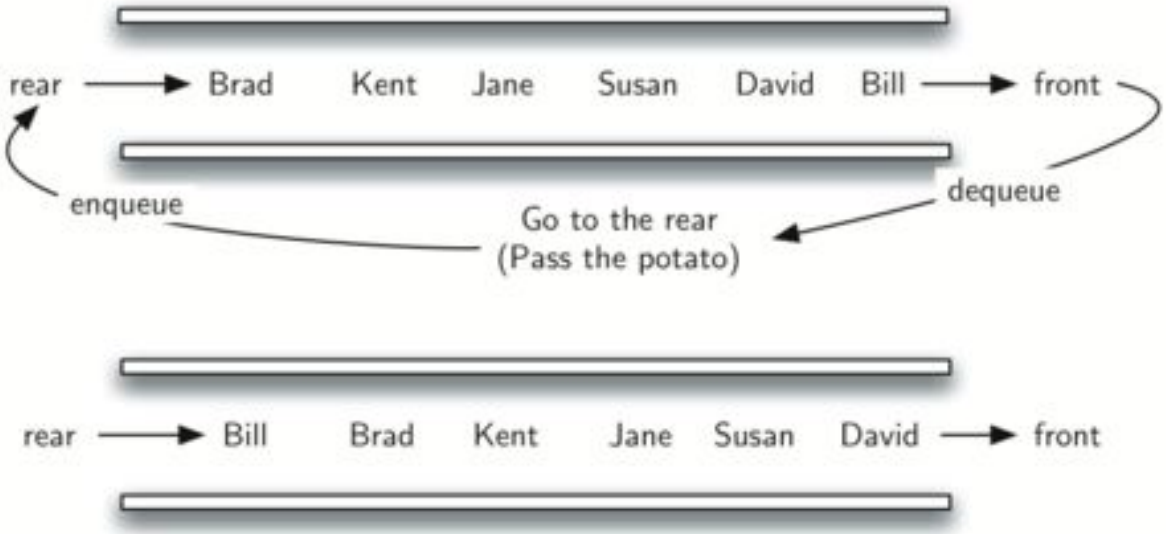
1. A ordem de chegada define a ordem de atendimento.
2. Idosos (60+), deficientes, grávidas e pessoas com crianças de colo têm prioridade.
3. Idosos (80+) têm maior prioridade.

Batata Quente

- Crianças se alinham em um círculo e passam a batata quente de vizinho para vizinho o mais rápido que puderem.
- A um certo ponto no jogo, a ação é interrompida e a criança que tem o item a batata é removida do círculo.
- O jogo para quando resta apenas uma criança.



Batata Quente



Batata Quente

```
from pythonds.basic.queue import Queue

def hotPotato(namelist, num):
   simqueue = Queue()
   for name in namelist:
       simqueue.enqueue(name)

   while simqueue.size() > 1:
       for i in range(num):
           simqueue.enqueue(simqueue.dequeue())

       simqueue.dequeue()

   return simqueue.dequeue()

print(hotPotato(["Bill","David","Susan","Jane","Kent","Brad"],7))
```

Resumo

- TADs de pilha e fila.
- A complexidade de algumas estruturas são diferentes.
- Pilhas e filas podem ser utilizadas para soluções de problemas reais.